

AI 原生(AI-Native)· 概念辨析与工程实践

编制:杨叶 · 用途:厘清「AI 原生」这个被用得极泛的词,并对照自己的 AI 安全测试平台做定位

一句话结论:AI 原生不是「用了 AI」,而是「把 AI 拿掉,这个产品就不成立」。

目录

目录	1
第 1 章 · AI 原生到底是什么	3
1.1 定义	3
1.2 三个层次	3
1.3 与「AI 赋能」的分界(四个判据)	3
1.4 判定清单	4
第 2 章 · AI 原生应用的架构	5
2.1 范式变化:从确定性到概率性	5
2.2 分层架构	5
2.3 三档形态:不要一上来就做 Agent	6
2.4 Agent 的四种构建方式	6
2.5 MCP —— 让工具接入标准化	7
2.6 上下文工程 —— AI 原生真正的功夫	7
2.7 评测体系 —— 最容易缺、也最致命的一环	8
2.8 护栏与人在环	9
第 3 章 · AI 原生开发	10
3.1 什么叫 AI 原生开发	10

3.2 四个阶段	10
3.3 工程约束(想上 L3 必须先有)	10
3.4 对研发流程的冲击(与安全直接相关)	10
第 4 章 · AI 原生安全	12
4.1 两个方向,别混为一谈	12
4.2 Security for AI —— 保护 AI 系统	12
4.3 AI for Security —— 用 AI 做安全	12
4.4 Agent 特有的安全风险(最新、也最容易问)	13
第 5 章 · 对照自己的平台	14
5.1 已经是 AI 原生的部分	14
5.2 还欠缺、但正好可以作为"下一步"来讲的部分	14
5.3 面试怎么讲(三段式)	14
第 6 章 · 术语与事实速查	16
6.1 术语表	16
6.2 容易说错的事实(现场别踩)	16

第 1 章 • AI 原生到底是什么

1.1 定义

AI 原生(AI-Native)指的是:系统的核心工作流由模型推理驱动,而不是由预先写死的确定性逻辑驱动。

这个词现在被滥用到几乎失去意义——加了个聊天框就叫 AI 原生。真正的分界线只有一条:

把模型能力拿掉,这个产品还剩下什么?

>

- 还剩一个能用的产品 → 那是 **AI 赋能(AI+)**
- 什么都不剩,或者退化成一堆没人用的工具 → 那才是 **AI 原生**

1.2 三个层次

「AI 原生」在不同语境下指三件不同的事,面试时要先问清楚对方问的是哪一层:

层次	指什么	典型问题
AI 原生应用	产品形态本身以模型推理为核心	架构怎么设计?怎么处理不确定性?
AI 原生开发	研发方式以 AI 协作为默认	代码怎么写?怎么保证质量?
AI 原生组织 / 基础设施	团队结构与平台围绕 AI 重建	谁来做?成本怎么算?治理怎么管?

- 招聘 JD 里说的「AI 原生」**九成指前两层**,架构岗偏第一层,研发效能岗偏第二层。

1.3 与「AI 赋能」的分界(四个判据)

维度	AI 赋能(AI+)	AI 原生(AI-Native)
AI 的位置	挂在既有产品旁边的一个功能	主链路本身
失去 AI 后	产品仍然可用,只是少个功能	产品不成立
不确定性	视为异常,尽量回避	视为常态,架构里专门处理 (评测、护栏、回退、人在环)
核心资产	业务代码与数据库	上下文与评测集 ——提示、检索语料、工具定义、评测用例

一个具体例子(用你熟悉的场景):

- **AI 赋能的安全测试:**原有扫描器不变,末尾加一个「用大模型润色报告」——去掉 AI,扫描器照常工作。
- **AI 原生的安全测试:**接口发现、攻击面推理、漏洞真伪判定本身由模型完成——去掉 AI,平台就只剩一堆脚本,3000+ 接口没人能梳理。

1.4 判定清单

一个系统满足以下**四条中的三条**,才配叫 AI 原生:

- ☐ **主链路依赖推理:**核心决策由模型做出,而不是模型只做辅助润色
- ☐ **处理不确定性是架构的一部分:**有评测集、有护栏、有失败回退、有人在环节点
- ☐ **上下文是一等公民:**有专门的上下文组织、检索、压缩、缓存策略,而不是把一堆文本塞进提示词
- ☐ **能力随模型升级而升级:**换一个更强的模型,产品能力直接变强,而不需要重写业务逻辑
- **最后一条是最狠的试金石:**如果换模型对产品没影响,那模型在你这里只是个可替换的字符串处理器。

第 2 章 • AI 原生应用的架构

2.1 范式变化:从确定性到概率性

	传统应用	AI 原生应用
执行逻辑	确定性——同样输入必然同样输出	概率性 ——同样输入可能不同输出
正确性保障	单元测试断言精确值	评测集 + 阈值 ,看统计质量而非单点相等
失败处理	抛异常	降级、重试、换路径、转人工
性能瓶颈	CPU / IO	上下文长度、token 成本、推理延迟
版本管理	代码版本	代码 + 提示 + 模型版本 + 评测集 四者一起版本化

- ⚠ **最容易被忽略的一条是最后一行**:很多团队把提示词硬编码在代码里、模型 ID 写死、没有评测集,结果模型一升级就不知道是变好还是变坏。

2.2 分层架构

一个成熟的 AI 原生应用大致分六层,从下往上:

层	内容
模型层	模型选型与路由(强模型做判断、轻模型做批量)、思考深度与成本档位、失败回退
上下文层	提示组织、检索(RAG)、上下文压缩与裁剪、缓存、长期记忆
工具层	工具定义与描述、参数 schema、工具检索、权限与副作用控制
编排层	单次调用 / 工作流 / Agent 循环;多智能体分工;并发与预算控制
护栏层	输入过滤、输出校验、结构化约束、越权与注入防护、人在环审批
观测与评测层	轨迹留存、token 与成本核算、质量评测、回归、A/B

- 判断一个团队是不是真做过 AI 原生,看最上面两层有没有。 底下四层是"能跑起来",上面两层才是"能上生产"。

2.3 三档形态:不要一上来就做 Agent

档位	形态	适用	特点
① 单次调用	一次请求一次响应	分类、摘要、抽取、问答	最简单、最可控、最便宜
② 工作流	你写编排逻辑,模型做其中若干步	多步流水线、步骤已知	控制流在你手里 ,可预测、好调试
③ Agent	模型自己决定调用什么工具、循环几轮	任务开放、路径无法预先穷举	能力最强, 但成本、延迟、不可预测性都最高

要不要上 Agent,问四个问题:

1. **复杂度**——任务是不是多步且无法预先完整描述?(「把这份设计文档变成一个 PR」是,「从 PDF 里抽标题」不是)
 2. **价值**——结果值不值更高的成本和延迟?
 3. **可行性**——模型在这类任务上到底行不行?
 4. **纠错成本**——错了能不能被发现和回滚?(有测试、有评审、有回滚)
- **任何一条答"否",就退回工作流或单次调用。** 现实中大量项目失败的原因是「本该是工作流的东西做成了 Agent」。

2.4 Agent 的四种构建方式

区分它们的关键是两个独立问题:**谁提供 harness(循环 + 上下文管理)**,**谁提供部署(运行的基础设施)**。

方式	你写什么	harness / 部署	适用
① 手写循环	自己写「模型要调工具 → 执行 → 回填 → 再问」的循环	都自己来	要完全掌控控制流
② SDK 的 Tool Runner	只写工具函数	SDK 给 harness,你自己部署	大多数自定义工具 Agent。仍有每轮钩子做审批、拦截、重试
③ 托管 Agent(Managed Agents)	Agent 配置 + 工具结果	平台给 harness,平台也托管每会话沙箱	要平台跑循环并托管工作区;长会话、定时任务、持久化配置

方式	你写什么	harness / 部署	适用
④ Agent SDK(如 Claude Agent SDK)	一个提示 + 若干选项	SDK 给完整编码 harness 和 内置工具,你自己部署	想要开箱即用的编码 / 文件系统 Agent

- **△ 最常见的混淆:**「Tool Runner」和「Agent SDK」听起来像一回事,其实完全不同——前者只是 API SDK 里帮你跑工具循环的薄封装(没有内置工具),后者是把编码智能体整套打包成库(自带读写文件、执行命令、搜索等工具)。
- **④ 正是你平台底座的形态**——Claude Code 类工具就是这一档:自带编码 harness 与内置工具,你在自己的机器上跑。

2.5 MCP —— 让工具接入标准化

MCP(Model Context Protocol) 是一个开放协议,用来统一「模型 ↔ 外部工具 / 数据源」的接入方式。

它解决的问题:在 MCP 之前,每接一个工具就要写一遍适配——不同的鉴权、不同的调用约定、不同的结果格式。**N 个模型 × M 个工具 = N×M 份适配代码。**

MCP 的做法:把工具方实现成 **MCP Server**,模型侧实现成 **MCP Client**,双方按协议对话。**N+M 取代 N×M。**

对 AI 原生系统的意义:

- **工具生态可复用**——一个 MCP Server 可被任意支持 MCP 的宿主使用
- **能力边界可管理**——工具的暴露范围、权限、审批点集中在协议层控制
- **△ 安全含义(你最该关注的):**MCP Server 是**新的信任边界**。恶意或被攻陷的 MCP Server 可以返回带有**间接提示注入**的内容,诱导模型执行非预期动作。见 4.4。

2.6 上下文工程 —— AI 原生真正的功夫

「提示词工程」是 2023 年的说法;现在的核心命题是「上下文工程」——如何在有限的上下文窗口和成本内,把最相关的信息以最有效的结构送进模型。

即便当前主流模型的上下文窗口已经到**百万 token 量级**,上下文工程依然是核心,原因有三:**成本随 token 线性增长、注意力随长度稀释、延迟随长度上升。**

五种手段,按使用顺序:

手段	做什么	关键点
----	-----	-----

手段	做什么	关键点
① 检索(RAG)	只把相关片段放进上下文	切分策略、混合检索、重排序,召回质量决定上限
② 提示缓存	稳定前缀命中缓存,大幅降本	前缀匹配 ——前缀里任何一个字节变化都会让后面全部失效。稳定内容放前面,时间戳、随机 ID 这类易变内容放后面
③ 上下文编辑	清除旧的工具结果或思考块	是「删」不是「摘要」,适合工具调用产生大量中间结果的场景
④ 压缩(Compaction)	把早期对话摘要后替换	是「摘要」不是「删」,适合长会话
⑤ 记忆(Memory)	跨会话保留知识	需要设计写入策略与失效策略,否则会积累错误记忆

- ⚠ **提示缓存最常见的坑**:系统提示里塞了 `now()`、每次请求带随机 ID、工具列表顺序不固定、JSON 序列化没排序——**这些都会静默让缓存命中率归零**。判断方法:看返回里的缓存读取 token 数,如果重复请求下一直是 0,就是有静默失效源。

2.7 评测体系 —— 最容易缺、也最致命的一环

没有评测集的 AI 原生系统,等于没有测试的软件。

层级	评什么	方法
单点质量	单次输出对不对	黄金集 + 精确匹配 / 结构校验
主观质量	好不好、有没有用	LLM-as-judge (用模型评模型)+ 人工抽检校准
端到端任务	整个任务完成没有	任务成功率、步数、成本、耗时
回归	改了提示 / 换了模型有没有变差	每次变更跑全量评测集并比对基线
成本与延迟	贵不贵、慢不慢	token 分布、P95 延迟、单任务成本

- **三条实践原则**:
 1. **评测集要先于优化存在**——没有基线就没有"改进"这回事
 2. **LLM-as-judge 必须做人工校准**——用一批人工标注样本验证评委模型的一致性,否则是自说自话
 3. **提示、模型版本、评测集三者一起进版本控制**——任何一个变了都要重跑

2.8 护栏与人在环

护栏类型	做什么
输入侧	过滤明显恶意输入;标注不可信来源(用户输入、抓取内容、工具返回)
输出侧	结构化输出约束 (用 schema 强制格式)、敏感信息过滤、事实性校验
动作侧	高危动作需人工审批 (删除、支付、对外发布);工具权限最小化;副作用可回滚
循环侧	最大轮数、 任务预算(token 上限) 、超时、死循环检测

- **人在环(Human-in-the-loop)不是失败,是设计。**判断放在哪:**动作的不可逆程度 × 错误代价**。可逆且低代价的自动放行,不可逆或高代价的必须人工确认。

第 3 章 • AI 原生开发

3.1 什么叫 AI 原生开发

不是「写代码时用 AI 补全」,而是「默认由 AI 产出、人来定义问题和把关结果」。

分界线同样清晰:AI 补全是你写主体、AI 帮你打字;AI 原生开发是你写规格与验收标准、AI 写实现、你审查。

3.2 四个阶段

阶段	形态	人的角色
L1 补全	行内代码补全	写代码的人
L2 对话	在聊天里问方案、生成片段,手工粘贴	写代码的人 + 提问的人
L3 委托	交给智能体一个任务,它读代码库、改多个文件、跑测试	定义问题 + 审查结果
L4 编排	多个智能体并行处理任务队列,人只看汇总与异常	设计流程 + 定验收标准

- 大多数团队卡在 L2。从 L2 到 L3 的门槛不是模型能力,是**工程可验证性**——没有测试、没有类型、没有明确验收标准的代码库,委托出去的结果无法自动判定对错。

3.3 工程约束(想上 L3 必须先有)

- **可验证**——有测试、有类型检查、有 lint;智能体能自己跑、自己判断成没成
- **小步提交**——一次改动可独立评审与回滚,不要一次生成两千行
- **规格先行**——先写清楚验收标准再委托,而不是让模型猜你要什么
- **强制审查**——AI 产出的代码和人写的走同一套评审与安全卡点,不能开绿色通道
- **上下文可得**——项目约定、架构说明、领域术语要以文档形式存在于代码库里,否则智能体每次都在猜

3.4 对研发流程的冲击(与安全直接相关)

变化	安全侧的含义
代码产出量激增	人工代码审查成为瓶颈 ——必须靠自动化安全检测顶上

变化	安全侧的含义
相似模式大规模复制	一个不安全写法会被 批量复制到很多处 ;反过来,一条检测规则也能批量拦截
依赖引入更随意	供应链风险上升 ——需要 SBOM、许可证审查、组件白名单
生成代码来源不透明	需要 AI 生成代码的标记与追溯 ,以及针对性的安全扫描
智能体拥有执行权限	凭据管理与权限最小化 成为硬需求;智能体的操作要留痕可审计

- **这张表是你把"AI 原生开发"接回安全主线的桥**:AI 原生开发不是替代安全测试,而是**让安全测试的自动化程度必须同步提升**,否则安全就变成新的瓶颈。

第 4 章 • AI 原生安全

4.1 两个方向,别混为一谈

方向	含义	你的位置
Security FOR AI	保护 AI 系统本身的安全	你有 OWASP LLM Top 10 检测能力
AI FOR Security	用 AI 做安全工作	你的平台属于这一类

- 面试被问「你怎么看 AI 安全」时,先做这个区分再回答,能立刻显出层次。

4.2 Security for AI —— 保护 AI 系统

核心风险清单参照 OWASP LLM Top 10,几类最关键的:

风险	说明	防护要点
提示注入	直接注入(用户输入)与 间接注入 (模型读到的网页、文档、工具返回里藏指令)	标注不可信来源、指令与数据分离、输出侧动作校验、 不让模型输出直接触发高危动作
敏感信息泄露	训练数据、系统提示、其他租户数据经模型泄露	最小化上下文、输出过滤、租户隔离、系统提示不放机密
不安全输出处理	模型输出被下游直接当代码 / SQL / 命令执行	把模型输出当作不可信输入——这是最重要的一条
供应链	模型、插件、MCP Server、第三方工具被投毒	来源校验、签名、权限最小化、沙箱
越权与过度代理	智能体权限过大,能做超出预期的事	工具权限最小化、审批点、副作用可回滚
资源滥用	无限循环、超长上下文、成本失控	轮数上限、 任务预算 、超时、限流

- ⚠ 一句能体现深度的话:「**间接提示注入是 AI 原生系统最本质的新增攻击面——因为模型无法在语义层可靠区分『数据』和『指令』**,所以防线必须放在输出侧的动作校验,而不是输入侧的关键词过滤。」

4.3 AI for Security —— 用 AI 做安全

AI 在安全工作里最有价值的三类用途,恰好都在你的平台里:

用途	为什么 AI 适合
理解非结构化产物	读源码、还原接口、理解业务语义——传统规则做不了
降噪与复核	扫描器误报率高, AI 做可利用性复核能大幅降低人工成本
知识沉淀与迁移	把一次分析的结论转成可复用的检测规则与用例

同时要清醒的三条边界:

1. **AI 不做最终判定**——高危结论必须人工复核, 这是可信度的来源
2. **AI 的产出需要评测**——误报率从多少降到多少, 得有统计口径, 不能凭感觉
3. **AI 不解决覆盖度问题**——覆盖度靠资产清单和用例体系, 不靠模型聪明

4.4 Agent 特有的安全风险(最新、也最容易问)

传统应用安全的框架不完全适用于智能体, 多出四类风险:

风险	说明
工具滥用	模型被诱导调用了不该调的工具, 或用了不该用的参数
权限累积	多轮交互后智能体累积了超出单步所需的权限与上下文
信任边界模糊	工具返回的内容进入上下文后, 与用户指令混在同一个语义空间
可审计性缺失	智能体做了什么、为什么这么做, 如果没有轨迹留存就无法追责

对应的防护设计:

- **工具最小权限 + 参数白名单 + 高危工具强制审批**
- **不可信内容标注**——工具返回、抓取内容进入上下文时明确标记来源
- **动作校验在输出侧**——不管模型说什么, 执行前按规则再校验一次
- **全量轨迹留存**——每次工具调用的输入输出、模型决策依据都要可回放
- **沙箱与网络隔离**——智能体的执行环境不接生产、不放真实凭据、出网走白名单

第 5 章 • 对照自己的平台

5.1 已经是 AI 原生的部分

对照 1.4 的判定清单,你的「源代码 → API 接口全链路自动化安全测试平台」:

判据	是否满足	依据
主链路依赖推理	☑	接口逆向、攻击面推理、漏洞真伪复核由模型完成——去掉 AI 只剩脚本
上下文是一等公民	△ 部分	6 条产品线、约 80 万行代码必然做了切分与检索策略,但是否成体系需要能讲清楚
处理不确定性	△ 部分	有人工复核环节(人在环),但评测体系是否成型是关键
能力随模型升级	☑	底座换更强的模型,检测与复核能力直接提升

• 结论:主链路上是 AI 原生的,工程完备度上还差评测与上下文工程的体系化表达。

5.2 还欠缺、但正好可以作为"下一步"来讲的部分

这三点被问到时主动说出来,比假装完美更有说服力:

1. 评测体系——「误报率从 60%+ 降到 12%」目前是抽样人工判定;更完整的做法是建立固定评测集 + 回归,每次提示或模型变更都跑一遍,才能区分「改进」和「运气」。
2. 上下文工程的体系化——大仓库怎么切分、检索怎么召回、缓存前缀怎么设计,这些如果只是"能跑",还没到"可优化"。
3. 平台自身的安全——这是最容易被忽略、也最能体现你安全背景的一点:平台读取的是源码和接口响应,全是不可信输入,存在间接提示注入风险;智能体又有执行权限。所以平台自己就该按 4.4 的方式做防护:工具最小权限、动作输出侧校验、轨迹留存、沙箱隔离。

5.3 面试怎么讲(三段式)

① 先定义,别让对方觉得你在蹭词

「我理解的 AI 原生不是『用了 AI』,而是**把 AI 拿掉这个产品就不成立**。按这个标准,我做的平台是 AI 原生的——接口逆向、攻击面推理、漏洞真伪判定这些主链路环节由模型完成,去掉模型就只剩一堆脚本,3000+ 接口根本梳理不过来。」

② 再讲工程,证明不是玩具

「AI 原生系统真正难的不是调通模型,是**处理不确定性**:所以我们保留了人工复核环节做最终判定,把每次分析的结论沉淀成可复用的检测规则,并且和 CI/CD 打通做常态化回归。」

③ 主动说边界,建立可信度

「我也很清楚它现在的短板:**评测体系还不够系统**,误报率的改善目前靠抽样人工判定,更严谨的做法是固定评测集加回归比对。另外**平台自己也是攻击面**——它读的源码和接口响应都是不可信输入,存在间接提示注入风险,这块的防护是我下一步要做的。」

- **第 ③ 段是最加分的**:它同时展示了 AI 工程认知和安全专业视角,而且主动暴露短板反而建立可信度。

第 6 章 • 术语与事实速查

6.1 术语表

术语	一句话解释
AI 原生	主链路由模型推理驱动;拿掉 AI 产品不成立
上下文工程	在有限窗口与成本内组织最有效信息的工程实践,已取代"提示词工程"成为核心命题
RAG	检索增强生成——先检索相关片段再让模型基于它作答
工具调用	模型请求执行外部函数,结果回填进上下文继续推理
Agent / 智能体	模型自主决定调什么工具、循环几轮,直到任务完成
MCP	模型上下文协议,统一「模型 ↔ 工具 / 数据源」接入,把 N×M 适配变成 N+M
提示缓存	稳定前缀命中缓存以降本; 前缀匹配,一字节变化即全部失效
上下文编辑	清除 旧工具结果或思考块(删)
压缩 Compaction	把早期对话 摘要 后替换(摘)
记忆 Memory	跨会话保留知识
结构化输出	用 schema 约束模型输出格式,保证可被程序消费
LLM-as-judge	用模型给模型的输出打分,需人工校准
护栏 Guardrail	输入 / 输出 / 动作 / 循环四个位置的安全与质量约束
人在环 HITL	在不可逆或高代价动作前插入人工确认
间接提示注入	恶意指令藏在模型读取的数据里(网页、文档、工具返回),而非用户直接输入

6.2 容易说错的事实(现场别踩)

✗过时或错误	☑当前准确说法
「上下文窗口就几万 token,所以必须拼命压缩」	主流前沿模型上下文已达 百万 token 量级 。压缩的理由变成了 成本、注意力稀释与延迟 ,不是塞不下

✗过时或错误	☑当前准确说法
「思考要设固定 token 预算」	固定思考预算的做法已被 自适应思考 + 努力档位(effort) 取代; 在新模型上传固定预算参数会直接报错
「Tool Runner 就是 Agent SDK」	两者不同: Tool Runner 是 API SDK 里帮你跑工具循环的薄封装(无内置工具);Agent SDK 是把编码智能体整套打包成库(自带文件读写、执行命令等内置工具)
「用了 Agent 就是 AI 原生」	Agent 只是形态之一。 大量场景该用工作流而不是 Agent ——上 Agent 前先过 2.3 的四问
「MCP 是某一家的私有接口」	MCP 是 开放协议 ,目的是让工具生态跨宿主复用
「提示词工程是核心竞争力」	现在的核心是 上下文工程 + 评测体系 ;单纯的提示技巧壁垒很低
「AI 输出可以直接执行」	模型输出必须当作不可信输入 处理,这是 AI 原生安全的第一原则